Unit 3

Experiment 9 - CI/CD Pipeline with Testing & Deployment

Aim:

Build a complete CI/CD pipeline that includes automated testing, containerized environments, contract validation, and staged deployment strategies.

Description:

In modern software development, delivering applications quickly and reliably is essential. Manual testing and deployment are slow, error-prone, and not scalable. To solve this, organizations use CI/CD pipelines.

This experiment introduces a complete pipeline including:

- Automated testing (unit, integration, contract)
- Real infrastructure testing (Testcontainers)
- Containerization using Docker
- Automation using GitHub Actions
- Deployment strategies (Canary Deployment)

Theoretical Explanation:

1. **Continuous Integration (CI)**

   Continuous Integration is the practice of automatically building and testing code whenever changes are pushed to the repository.

   How CI Works?

   Developer pushes code → CI Triggered → Build → Test → Result

   1. Code is pushed to GitHub
   2. GitHub Actions detects the push
   3. A virtual machine (runner) starts
   4. Code is downloaded
   5. Build command runs (mvn clean install)
   6. Tests execute
   7. Pipeline passes or fails

   Why CI is important?

   - Detect bugs early
   - Ensure code quality
   - Reduce integration issues

   When to Use?

   - Team-based development
   - Frequent code updates

   Advantages:

   - Faster feedback

- Reduced bugs
- Improved collaboration

Drawbacks:

- Requires proper test coverage → Weak tests = false confidence
- Initial setup effort → Requires pipeline configuration

8. Continuous Deployment (CD)

Continuous Deployment automatically deploys the application after successful testing.

Why CD?

- Faster delivery
- Eliminates manual deployment

How CD Works?

CI passes → application is deployed automatically

When to Use?

- Mature systems
- High test reliability

Advantages:

- Rapid releases
- Consistent deployments

Drawbacks:

- Risk if tests are weak
- Requires monitoring

9. Unit Testing (JUnit 5)

Testing individual methods or functions.

Why?

- Validate logic correctness
- Catch bugs early

Flow:

Input → Function → Output → Compare expected

How? (Example)

```
@Test
void testAdd() {
    int result = calculator.add(2, 3);
    assertEquals(5, result);
}
```

Detailed Explanation:

- `@Test` → marks method as test
- `calculator.add(2,3)` → actual logic
- `assertEquals(5, result)` → verifies output

If output ≠ expected → test fails

Advantages:

- Fast execution
- Isolated testing

Drawbacks:

- Does not test full system

10. Integration Testing (MockMvc)

Testing APIs by simulating HTTP requests.

Need: Ensures components work together

Example:

```
@Test
void testGetPosts() throws Exception {
    mockMvc.perform(get("/posts"))
           .andExpect(status().isOk());
}
```

Explanation:

- `perform()` → sends request
- `get("/posts")` → API endpoint
- `isOk()` → checks HTTP 200

Advantages:

- Tests real behavior
- Covers multiple layers

Drawbacks: Slower than unit tests

11. Contract Testing (Pact)

Ensures frontend and backend agree on API format.

Problem:

Frontend expects – `{ "name": "XYZ" }`

Backend sends - `{ "username": "XYZ" }`

Solution: Contract Testing that ensures both sides agree on API Structure

How

- Define expected request/response
- Validate backend against it

When to do?

- When multiple teams work independently

Advantages:

- Reduces communication issues
- Ensures compatibility

Drawbacks:

- Additional setup effort

12. Testcontainers

Runs real services (DB/Kafka) in Docker during tests.

Need: More realistic than mocks

How

```
@Container
static PostgreSQLContainer<?> db =
    new PostgreSQLContainer<>("postgres:15");
```

Explanation:

- Docker container starts DB
- Tests run on real DB
- Container stops automatically

Advantages:

- Real environment testing
- High reliability

Drawbacks:

- Slower execution
- Requires Docker

13. Docker & Containerization

Packaging application with dependencies.

Why to use?

- Runs same everywhere
- Avoids environment issues

Example:

```
FROM openjdk:17
COPY target/app.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Explanation:

- Base image → Java
- Copy → application
- Run → executes app

Advantages:

- Portability
- Consistency

Drawbacks:

- Learning curve
- Resource overhead

14. GitHub Actions

Automation tool for CI/CD pipelines.

Why to use?

- Automates testing & deployment

Example:

```
name: CI Pipeline

on:
  push:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: mvn clean install
```

Explanation:

- Trigger → push
- Steps → build project
- Automates workflow

When – while implementing Continuous integration (CI) pipeline

Advantages:

- Automation
- Easy integration

Drawbacks:

- YAML complexity
- Debugging pipelines

15. Canary Deployment

Deploy to a small set of users first.

Why: Reduce risk

How?

- Release to 10% users
- Monitor
- Expand rollout

When?

- Production deployments

Advantages:

- Safe deployment
- Early bug detection

Drawbacks:

- Requires monitoring setup
- Slightly complex setup

Code implementation

Github Actions CI Pipeline

```
Complete CI Workflow
name: CI Pipeline

on:
  push:
    branches: [ "main" ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v3

      - name: Set up Java
        uses: actions/setup-java@v3
        with:
          distribution: 'temurin'
          java-version: '17'

      - name: Build Project
        run: mvn clean install

      - name: Run Tests
        run: mvn test
```

Step-by-step Execution –

1. Trigger – code pushed to branch “main”
2. Runner starts – Github creates a fresh VM
3. Checkout code – Downloads your repo
4. Setup Java – Installs Java 17
5. Build – Compiles project
6. Testing – Runs unit + integration tests

If any test fails → pipeline stops

Continuous Deployment (CD) with Docker

```
name: CD Pipeline

on:
  workflow_run:
    workflows: ["CI Pipeline"]
    types:
      - completed

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
```

```
    uses: actions/checkout@v3

  - name: Build Docker Image
    run: docker build -t myapp .

  - name: Login to DockerHub
    run: echo "PASSWORD" | docker login -u USERNAME --password-stdin

  - name: Push Image
    run: docker push myapp

  - name: Deploy (Simulated)
    run: echo "Deploying application..."
```

Step-by-step Execution –

1. Trigger – runs only after CI completes successfully
2. Build Image – Creates container image
3. Push Image – Uploads to DockerHub
4. Deployment – Server pulls latest image and runs container

<u>Full CI/CD Flow</u>

Developer Push → CI Pipeline → Tests Pass → Build Docker Image → Push Image → Deploy → Monitor → Gradual Rollout

Experiment 9.1: Continuous Integration Pipeline Setup

Conceptual Focus:

- Unit testing
- Integration testing
- Contract testing
- Testcontainers
- CI automation

Implementation Overview:

- Write tests
- Configure pipeline
- Run tests automatically

Experiment 9.2: Continuous Deployment & Release Strategies

Conceptual Focus:

- Docker
- Deployment pipelines
- Canary deployment

Implementation Overview:

- Containerize app
- Deploy automatically
- Implement release strategy

Integration Perspective

CI/CD integrates testing, containerization, and deployment into a single automated workflow, enabling efficient, reliable, and scalable software delivery.

Assignments

Assignment 1 – Write Unit Tests

Assignment 2 – Write API Tests

Assignment 3 – Use Test Containers

Assignment 4 – Dockerize App

Assignment 5 – Implement canary deployment

Key Takeaways

- CI/CD automates development lifecycle
- Testing ensures reliability
- Docker ensures portability
- Deployment strategies reduce risk

Future Scope

Students can extend this to **Kubernetes, Terraform, cloud CI/CD pipelines, and enterprise DevOps systems**.